Documenting a C Library

Kevin Browne

How can we document our code?

- We can put comments in our code to explain things
  - Comments are then right next to what they document

- We can create an external document in a word processor (e.g. Word) or other system (LaTeX, HTML) external to our code
  - We can then create things like navigation or "tables of contents" that allow us to overview functionalities
  - We can document multiple code files in one document
  - We can use formatting to make content more readable

How can we document our code?

- Comments and an external document both have pros and cons
  - Comments document code in-place
  - External documents can summarize, format, allow navigation (e.g. hyperlinks), and illustrate connections
  - But producing both individually would involve reproducing content... then what if we need to make changes later? Do we change it in two places?

- Wouldn't it be nice if we could have the best of both worlds?
  - We can with document generator tools like Doxygen!

Document generators

- **Document generators** create software documentation from source code files
  - Documentation is typically in a format such as HTML, LaTeX, PDF, Word, etc.
  - Source code files are typically annotated with special comments intended to be processed by the document generator
  - The special comments indicate precisely what and how the document generator should capture certain features
  - Document generators can also recognize code features (functions, structs, files, etc.) and group documentation in a sensible way based on these features

Using Doxygen

- Doxygen is the defacto standard document generator for C/C++
- Available for Mac/Windows/Linux here:
  - https://www.doxygen.nl/index.html
- If we're going to comment our code anyways, why not comment it so that we can generate a nice readable document too?

Using Doxygen

- To use doxygen you need to first generate a doxygen configuration file ("Doxyfile")
- Done from the command line with: **doxygen -g**
- ...or can be done with GUI wizard too!

Doxygen configuration file

- Doxygen configuration file is massive, allows you to configure all kinds of things
  - **Important:** PROJECT_NAME should be set
  - Can also configure things like what directories to check for .c and .h files, what outputs to produce
  - By default, reads any .c and .h files in current directory

```
# The PROJECT_NAME tag is a single word (or a sequence of words surrounded by
# double-quotes, unless you are using Doxywizard) that should identify the
# project for which the documentation is generated. This name is used in the
# title of most generated pages and in a few other places.
# The default value is: My Project.

PROJECT_NAME           = "My Project"
```

Running Doxygen

- You can use the GUI interface, or the command-line interface
- Produces html and latex folders
- HTML folder contains website documentation, LaTeX folder contains latex file documentation

LaTeX

- **LaTeX** is the standard way to produce technical documents for computer scientists
- Plain text documents annotated with commands like **\begin{tabular} … \end{tabular}** (for producing tables) get compiled into formal documents in formats like PDF
- A bit similar to plaintext HTML documents being read by a web browser to produce web page layouts/structures

Documenting code with Doxygen

- We can write doxygen comments like this, with two stars at the top of the comment:

```
/**


*/
```

- This signals to doxygen that it should be looking at this comment to include in the documentation!

Documenting code with Doxygen

- When we place these comments above functions, Doxygen will document the function with that comment
  - Same for structs/typedefs

- Doxygen looks for specific commands that we can specify to produce documentation
  - Commands start with @
  - @param command documents function parameters
  - e.g. **@param length the length of the linked list**

Doxygen

- There are many standard comment formats that different document generators use
  - Doxygen supports just about all of them!
- For example, instead of @command for commands, we can use a \command
  - It really doesn't matter what you use so long as you stick with a format, they provide this for convenience
- See: https://www.doxygen.nl/manual/docblocks.html

Doxygen example

- Doxygen comments begin with **/****
- Comments are placed above typedef/structs, functions and files to document them
- Comments may include commands which denote how and what to document specifically

Doxygen commands

- **@file filename**
  - Should be in every file, documenting the filename
- **@author author name**
  - Documents the author of the file
- **@date date**
  - Documents the date the file was created (or perhaps last modified)

Doxygen commands

- **@brief description**
  - Can be used to provide a brief description of the function/file/etc.

- **@mainpage title**
  ….. text ……
  - Used to denote title and text for the main page of the documentation

Documenting a struct/typedef

```
/**
* Book type stores a book with fields title, author, isbn.
*
*/
typedef struct book {
   char title[50];  /**< the book's title */
   char author[50]; /**< the book's author */
   int isbn; /**< the book's 4 digit ISBN code */
} Book;
```

/**< description */ is used to provide a decription for each struct member

Documenting a function

```
/**
 * Prints all books in the library in a list incl. title, author, isbn
 *
 * @param library a library of books represented as an array.
 * @return nothing
 */
void print_library(Book library[])
{
  printf("\n****************************\n");
  for (int i = 0; i < NUMBER_OF_BOOKS; i++)
```

We can use @param to document parameters and @return to document return values